UNIT – I DATABASE SYSTEMS AND SQL QUERY

Introduction – Purpose of Database Systems - View of Data –Database Architecture - Database Schema– Keys – Codd's Rule –RDBMS–SQL: Data Definition – Domain types – Structure of SQL Queries – Modification of the database – Set Operation – Aggregate Function – Null Values – SQL Nested Subqueries – Views – Joined relations.

INTRODUCTION:

What is data?

- Data is nothing but facts and statistics stored or free flowing over a network, generally it's raw and unprocessed.
- Data becomes information when it is processed, turning it into something meaningful.

What is database?

- The database is a collection of inter-related data which is used to retrieve, insert and delete the data efficiently.
- It is also used to organize the data in the form of a table, schema, views, and reports, etc.
- Using the database, you can easily retrieve, insert, and delete the information.
- For example: The college Database organizes the data about the admin, staff, students and faculty etc.

What is dbms?

A database-management system (DBMS) consists of a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the database, contains information about one particular enterprise. The primary goal of a DBMS is to provide an environment that is both convenient and efficient to use in retrieving and storing database information.

The management of data involves both the definition of structures for the storage of information and the provision of mechanisms for the manipulation of information. The database system must also provide for the safety of the information stored, despite system crashes or attempts at unauthorized access.

PURPOSE OF DATABASE SYSTEMS :

The main purpose of database systems is to manage the data. Consider a university that keeps the data of students, teachers, courses, books etc. To manage this data we need to store this data somewhere where we can add new data, delete unused data, update outdated data, retrieve data, to perform these operations on data we need a Database management system that allows us to store the data in such a way so that all these operations can be performed on the data efficiently.

Characteristics of DBMS:

- **Data stored into Tables:** Data is never directly stored into the database. Data is stored into tables, created inside the database.
- **Reduced Redundancy:** In the modern world hard drives are very cheap, but earlier when hard drives were too expensive, unnecessary repetition of data in database was a big problem. But DBMS follows Normalisation which divides the data in such a way that repetition is minimum.
- **Data Consistency:** On Live data, i.e. data that is being continuosly updated and added, maintaining the consistency of data can become a challenge. But DBMS handles it all by itself.
- **Support Multiple user and Concurrent Access:** DBMS allows multiple users to work on it(update, insert, delete data) at the same time and still manages to maintain the data consistency.
- Query Language: DBMS provides users with a simple Query language, using which data can be easily fetched, inserted, deleted and updated in a database.

Advantages of DBMS:

- **Controls database redundancy:** It can control data redundancy because it stores all the data in one single database file and that recorded data is placed in the database.
- **Data sharing:** In DBMS, the authorized users of an organization can share the data among multiple users.
- **Easily Maintenance:** It can be easily maintainable due to the centralized nature of the database system.
- **Reduce time:** It reduces development time and maintenance need.
- **Backup:** It provides backup and recovery subsystems which create automatic backup of data from hardware and software failures and restores the data if required.
- **Multiple User Interface:** It provides different types of user interfaces like graphical user interfaces, application program interfaces

Disadvantages of DBMS:

- **Cost of Hardware and Software:** It requires a high speed of data processor and large memory size to run DBMS software.
- **Size:** It occupies a large space of disks and large memory to run them efficiently.
- **Complexity:** Database system creates additional complexity and requirements.
- **Higher impact of failure:** Failure is highly impacted the database because in most of the organization, all the data stored in a single database and if the database is damaged due to electric failure or database corruption then the data may be lost forever.

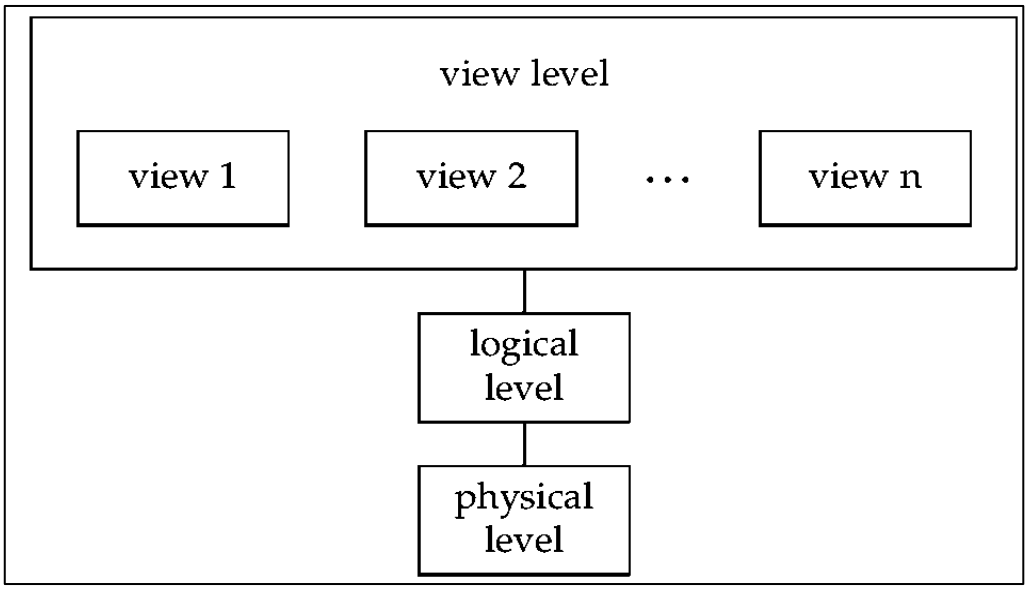
APPLICATIONS OF DBMS:

Applications where we use Database Management Systems are:

- **Telecom:** There is a database to keeps track of the information regarding calls made, network usage, customer details etc.
- **Industry:** Where it is a manufacturing unit, warehouse or distribution centre, each one needs a database to keep the records of ins and outs
- **Banking System:** For storing customer info, tracking day to day credit and debit transactions, generating bank statements etc.
- **Sales:** To store customer information, production information and invoice details.
- **Airlines:** To travel though airlines, we make early reservations; this reservation information along with flight schedule is stored in database.
- **Education sector:** Database systems are frequently used in schools and colleges to store and retrieve the data regarding student details, staff details, course details, exam details, payroll data, attendance details, fees details etc.

VIEW OF DATA

A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.



Data Abstraction: For the database system to be usable, it must retrieve data efficiently. This concern has led to the design of complex data structures for the representation of data in the database. Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify user‟s interactions with the system:

- **Physical level**: The lowest level of abstraction describes how the data are actually stored, which involves complex low-level data structures.

- **Logical level:** The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. The logical level of abstraction is used by database administrators, who must decide what information is to be kept in the database.

- **View level:** The highest level of abstraction describes only part of the entire database. Users need to access only a part of the database. So that their interaction with the system is simplified, the view level of abstraction is defined. The system may provide many views for the same database.

Instances and Schemas: Database change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database. The overall design of the database is called the database schema. Schemas are changed infrequently, if at all.

A database schema corresponds to the programming-language type definition. A variable of a given type has a particular value at a given instant. Thus, the value of a variable in programming languages corresponds to an instance of a database schema.

Database systems have several schemas. At the lowest level is the physical schema; at the intermediate level is the logical schema; and at the highest level is a subschema. In general, database systems support one physical schema, one logical schema, and several subschemas.

Data independence: The ability to modify a schema definition in one level without affecting a schema definition in the next higher level is called data independence. There are two levels of data independence:

1. Physical data independence is the ability to modify the physical schema without causing application programs to be rewritten. Modifications at the physical level are occasionally necessary to improve performance.

2. Logical data independence is the ability to modify the logical schema without causing application programs to be rewritten. Modifications at the logical level are necessary whenever the logical structure of the database is altered.

Logical data independence is more difficult to achieve than is physical data independence, since application programs are heavily dependent on the logical structure of the data that they access.

TYPES OF DBMS ARCHITECTURE

There are three types of DBMS architecture:

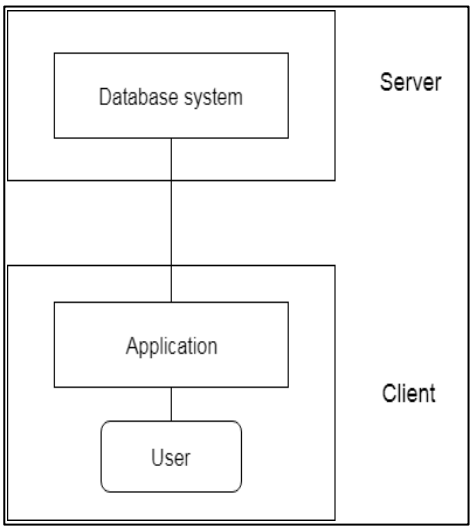
i. Single tier architecture
ii. Two tier architecture
iii. Three tier architecture

Single Tier Architecture:

- In this type of architecture, the database is readily available on the client machine, any request made by client doesn‟t require a network connection to perform the action on the database.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.

Two tier architecture:

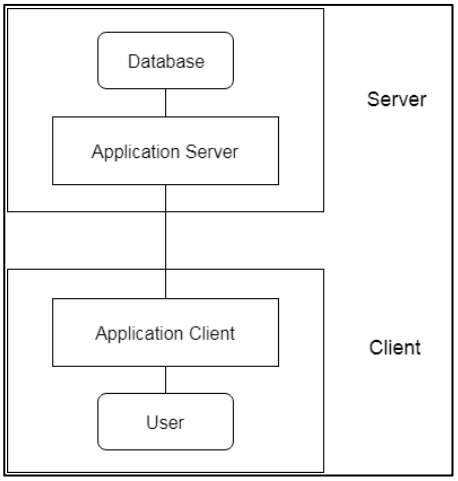
- In two-tier architecture, the Database system is present at the server machine and the DBMS application is present at the client machine, these two machines are connected with each other through a reliable network.
- Whenever client machine makes a request to access the database present at server using a query language like sql, the server perform the request on the database and returns the result back to the client.
- The application connection interface such as JDBC, ODBC are used for the interaction between server and client.



Three-Tier Architecture:

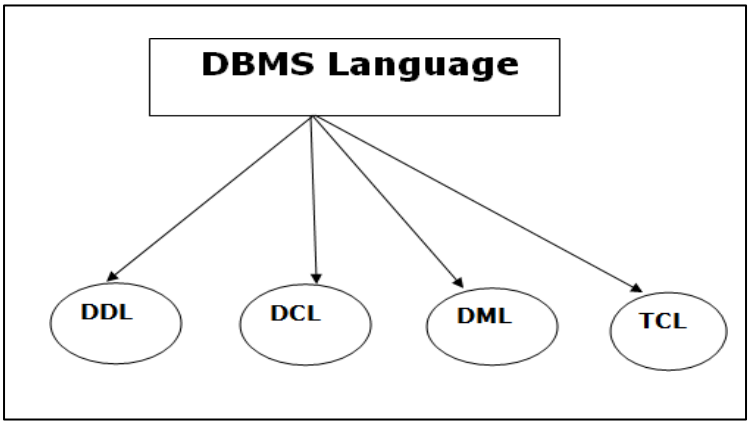
 In three-tier architecture, another layer is present between the client machine and server machine.

 In this architecture, the client application doesn't communicate directly with the database systems present at the server machine, rather the client application communicates with server application and the server application internally communicates with the database system present at the server.



DBMS languages:

Database languages are used to read, update and store data in a database. There are several such languages that can be used for this purpose; one of them is SQL (Structured Query Language).



- DDL – Data Definition Language: (CREATE,DROP,ALTER,TRUNCATE,COMMENT,RENAME)
- DML – Data Manipulation Language: (INSERT, UPDATE,DELETE)
- DCL – Data Control Language: (GRANT,REVOKE)
- TCL-Transaction Control Language: (COMMIT,ROLLBACK)

1. DDL(Data Definition Language) : DDL or Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database.
CREATE – it is used to create the database or its objects (like table, index, function, views, store procedure and triggers).
There are two CREATE statements available in SQL:
 CREATE DATABASE

 CREATE TABLE

CREATE DATABASE:
A Database is defined as a structured set of data. So, in SQL the very first step to store the data in a well structured manner is to create a database. The CREATE DATABASE statement is used to create a new database in SQL.
Syntax:
CREATE DATABASE database_name;
Example:
SQL> CREATE DATABASE Employee;

In order to get the list of all the databases, you can use SHOW DATABASES statement.
Example –
SQL> SHOW DATABASES;

CREATE TABLE:
The CREATE TABLE statement is used to create a table in SQL. We know that a table comprises of rows and columns. So while creating tables we have to provide all the information to SQL about the names of the columns, type of data to be stored in columns, size of the data etc. Let us now dive into details on how to use CREATE TABLE statement to create tables in SQL.

Syntax:
```
CREATE TABLE table_name (
column1   data_type(size),
column2   data_type(size),
column3  data_type(size),
....
);
```
Example Query: This query will create a table named Students with three columns, ROLL_NO, NAME and SUBJECT.
```
CREATE TABLE Students
(
ROLL_NO  int(3),
NAME varchar(20),
SUBJECT varchar(20),
);
```

DROP:
DROP is used to delete a whole database or just a table.The DROP statement destroys the objects like an existing database, table, index, or view.
A DROP statement in SQL removes a component from a relational database management system (RDBMS).
Syntax:
```
DROP object object_name;
```
Examples:
```
DROP  TABLE  table_name;
```
table_name: Name of the table to be deleted.
```
DROP DATABASE database_name;
```
database_name: Name of the database to be deleted.

TRUNCATE
It is used to remove all records from a table, including all spaces
The TRUNCATE TABLE mytable statement is logically (though not physically) equivalent to the DELETE FROM mytable statement (without a WHERE clause).
Syntax:
```
TRUNCATE  TABLE  table_name;
```

DROP vs TRUNCATE
- Truncate is normally ultra-fast and its ideal for deleting data from a temporary table.
- Truncate preserves the structure of the table for future use, unlike drop table where the table is deleted with its full structure.
- Table or Database deletion using DROP statement cannot be rolled back, so it must be used wisely.

To delete the whole database,
DROP DATABASE student_data;

After running the above query whole database will be deleted.

To truncate Student_details table from student_data database.
TRUNCATE TABLE Student_details;

DROP	TRUNCATE
The DROP command in SQL removes an entire table from a database including its definition, indexes, constraints, data etc.	The TRUNCATE command is used to remove all of the rows from a table, regardless of whether or not any conditions are met and resets the tabledefinition.
It is a DDL(Data DefinitionLanguage) command.	It is also a DDL(Data DefinitionLanguage) command.
The table space is completely freedfrom the memory.	The table still exists in the memory.
All the integrity constraints areremoved.	The integrity constraints still exist in thetable.
Requires ALTER and CONTROL permissions on the table schema and table respectively, to be able to perform this command.	Only requires the ALTER permissions totruncate the table.
DROP command is much slower than TRUNCATE but faster than DELETE.	It is faster than both DROP and DELETEcommands.

ALTER (ADD, DROP, MODIFY)

ALTER TABLE is used to add, delete/drop or modify columns in the existing table. It is also used to add and drop various constraints on the existing table.

ALTER TABLE – ADD: ADD is used to add columns into the existing table. Sometimes we may require to add additional information, in that case we do not require to create the whole database again, ADD comes to our rescue.

Syntax:
ALTER TABLE table_name
ADD (Columnname_1 datatype,
Columnname_2 datatype,
…
Columnname_n datatype);

ALTER TABLE – DROP:

DROP COLUMN is used to drop column in a table. Deleting the unwanted columns from the table.

Syntax:
ALTER TABLE table_name
DROP COLUMN column_name;

ALTER TABLE-MODIFY:

It is used to modify the existing columns in a table. Multiple columns can also be modified at once.

Syntax:
ALTER TABLE table_name
MODIFY column_name column_type;

QUERY:

To ADD 2 columns AGE and COURSE to table Student.
ALTER TABLE Student ADD (AGE number(3),COURSE varchar(40));

MODIFY column COURSE in table Student
ALTER TABLE Student MODIFY COURSE varchar(20);

Comments

As is any programming languages comments matter a lot in SQL also. In this set we will learn about writing comments in any SQL snippet.
Comments can be written in the following three formats:

- Single line comments.
- Multi line comments
- In line comments

DML(Data Manipulation Language) : The SQL commands that deals with the manipulation of data present in the database belong to DML or Data Manipulation Language and this includes most of the SQL statements.

SELECT Statement

SELECT statement is used to fetch data from relational database. A relational database is organized collection of data. As we know that data is stored inside tables in a database. SQL select statement or SQL select query is used to fetch data from one or more than one tables.

SELECT Syntax:

One column: Here column_name is the name of the column for which we need to fetch data and table_name is the name of the table in the database.

```
SELECT column_name FROM table_name;
```

More than one columns:

```
SELECT column_name_1, column_name_2, ... FROM table_name;
```

To fetch the entire table or all the fields in the table:

```
SELECT * FROM table_name;
```

Example:

```
SELECT EMP_NAME FROM EMPLOYEES;
```

To fetch the entire EMPLOYEES table:

```
SELECT * FROM EMPLOYEES;
```

Query to fetch the fields ROLL_NO, NAME, AGE from the table Student:

```
SELECT ROLL_NO, NAME, AGE FROM Student;
```

INSERT INTO Statement

The INSERT INTO statement of SQL is used to insert a new row in a table. There are two ways of using INSERT INTO statement for inserting rows:

Only values: First method is to specify only the value of data to be inserted without the column names.

```
INSERT INTO table_name VALUES (value1, value2, value3,…);
```

Column names and values both: In the second method we will specify both the columns which we want to fill and their corresponding values as shown below:

```
INSERT INTO table_name (column1, column2, column3,..) VALUES ( value1, value2, value3,..);
```

Example:

Method 1 (Inserting only values) :

```
INSERT INTO Student VALUES („5′,‟HARSH‟,‟WEST
BENGAL‟,‟XXXXXXXXXX‟,‟19‟);
```

Method 2 (Inserting values in only specified columns):

```
INSERT INTO Student (ROLL_NO, NAME, Age) VALUES („5′,‟PRATIK‟,‟19‟);
```

UPDATE Statement

The UPDATE statement in SQL is used to update the data of an existing table in database. We can update single columns as well as multiple columns using UPDATE statement as per our requirement.

Basic Syntax:

```
UPDATE TableName
SET column_name1 = value, column_name2 = value....
WHERE condition;
```

EX1:

```
SQL> UPDATE EMPLOYEES
SET EMP_SALARY = 10000
WHERE EMP_AGE > 25;
```

EX2:
SQL> UPDATE EMPLOYEES
SET EMP_SALARY = 120000
WHERE EMP_NAME = 'Apoorv';

DELETE Statement
The DELETE Statement in SQL is used to delete existing records from a table. We can delete a single record or multiple records depending on the condition we specify in the WHERE clause.
Basic Syntax:
DELETE FROM table_name WHERE some_condition;
Deleting single record: Delete the rows where NAME = „Ram". This will delete only the first row.
DELETE FROM Student WHERE NAME = 'Ram';

Deleting multiple records: Delete the rows from the table Student where Age is 20. This will delete 2 rows(third row and fifth row).
DELETE FROM Student WHERE Age = 20;

Delete all of the records: There are two queries to do this as shown below,
query1: "DELETE FROM Student";
query2: "DELETE * FROM Student";

DCL(Data Control Language) : DCL includes commands such as GRANT and REVOKE which mainly deals with the rights, permissions and other controls of the database system.
Examples of DCL commands:

GRANT-gives user"s access privileges to database.

REVOKE-withdraw user"s access privileges given by using the GRANT command.

TCL(transaction Control Language) : TCL commands deals with the transaction within the database.
Examples of TCL commands:

COMMIT– commits a Transaction.

ROLLBACK– rollbacks a transaction in case of any error occurs.

SAVEPOINT–sets a savepoint within a transaction.

SET TRANSACTION–specify characteristics for the transaction.

DATABASE USERS AND ADMINISTRATORS

Database Users and User Interfaces: There are four different types of database-system users.

- Application programmers are computer professionals who interact with the system through DML calls, which are embedded in a program written in a host language. These programs are referred to as application programs. A special preprocessor called the DML precompiler, converts the DML statements to normal procedure calls in the host language.

- Sophisticated users interact with the system without writing programs. Instead, they form their requests in a database query language. Each such query is submitted to a query processor whose function is to break down DML statement into instructions that the storage manager understands.

- Specialized users are sophisticated users to write specialized database applications that do not fit into the traditional data-processing framework. Among these applications are computer-aided design systems, knowledge base and expert systems, systems that store data with complex data types, and environment-modeling systems.

- Naive users interact with the system by invoking one of the permanent application programs that have been written previously.

Database Administrator: The person who has central control over the database system is called the database administrator (DBA). The functions of the DBA:

- Schema definition. The DBA creates the original database schema by writing a set of definitions that is translated by the DDL compiler to a set of tables that is stored permanently in the data dictionary.

- Storage structure and access-method definition. The DBA creates appropriate storage structures and access methods by writing a set of definitions, which is translated by the data-storage and data-definition-language compiler.

- Schema and physical-organization modification. The DBA carries out changes to the database schema and the physical storage organization by writing a set of definitions that is used by either the DDL compiler or the data-storage and data-definition-language compiler to generate modifications to the appropriate internal system tables.

- Granting of authorization for data access. The granting of different types of authorization allows the database administrator to regulate which parts of the database various users can access. This information is kept in a special system structure that is consulted by the database system whenever access to the data is attempted in the system.

- Integrity-constraint specification. The data values stored in the database must satisfy certain consistency constraints, and such constraints must be specified explicitly by the database administrator. The integrity constraints are kept in a special system structure that is consulted by the database system whenever an update takes place in the system.

TRANSACTION MANAGEMENT

A transaction is a collection of operations that performs a single logical function in a database application. A suitable example for a transaction is funds transfer, in which one account is debited and another account is credited.

Atomicity: A transaction must happen in it entirely in a database or not at all. This all-or-none requirement is called atomicity.

Consistency: The execution of a transaction must preserve the consistency of the database. It is required that transactions do not violate database consistency constraints. That is, if the database was consistent when a transaction started, the database must be consistent when the transaction successfully terminates it is the responsibility of the programmer to define properly the various transactions, such that each preserves the consistency of the database.

Durability: After the successful execution of a transaction, the new values must persist in the database, despite the possibility of system failure. This persistence requirement is called durability.

Each transaction is a unit of both atomicity and consistency. In case of system failures, a failed transaction should restore the database to the state that existed prior to the occurrence of the failure.

When several transactions update the database concurrently, the consistency of data may no longer be preserved, even though each individual transaction is correct. It is the responsibility of the concurrency-control manager to control the interaction among the concurrent transactions, to ensure the consistency of the database.

ACID Properties:

ACID Properties are used for maintaining the integrity of database during transaction processing. ACID in DBMS stands for Atomicity, Consistency, Isolation, and Durability.

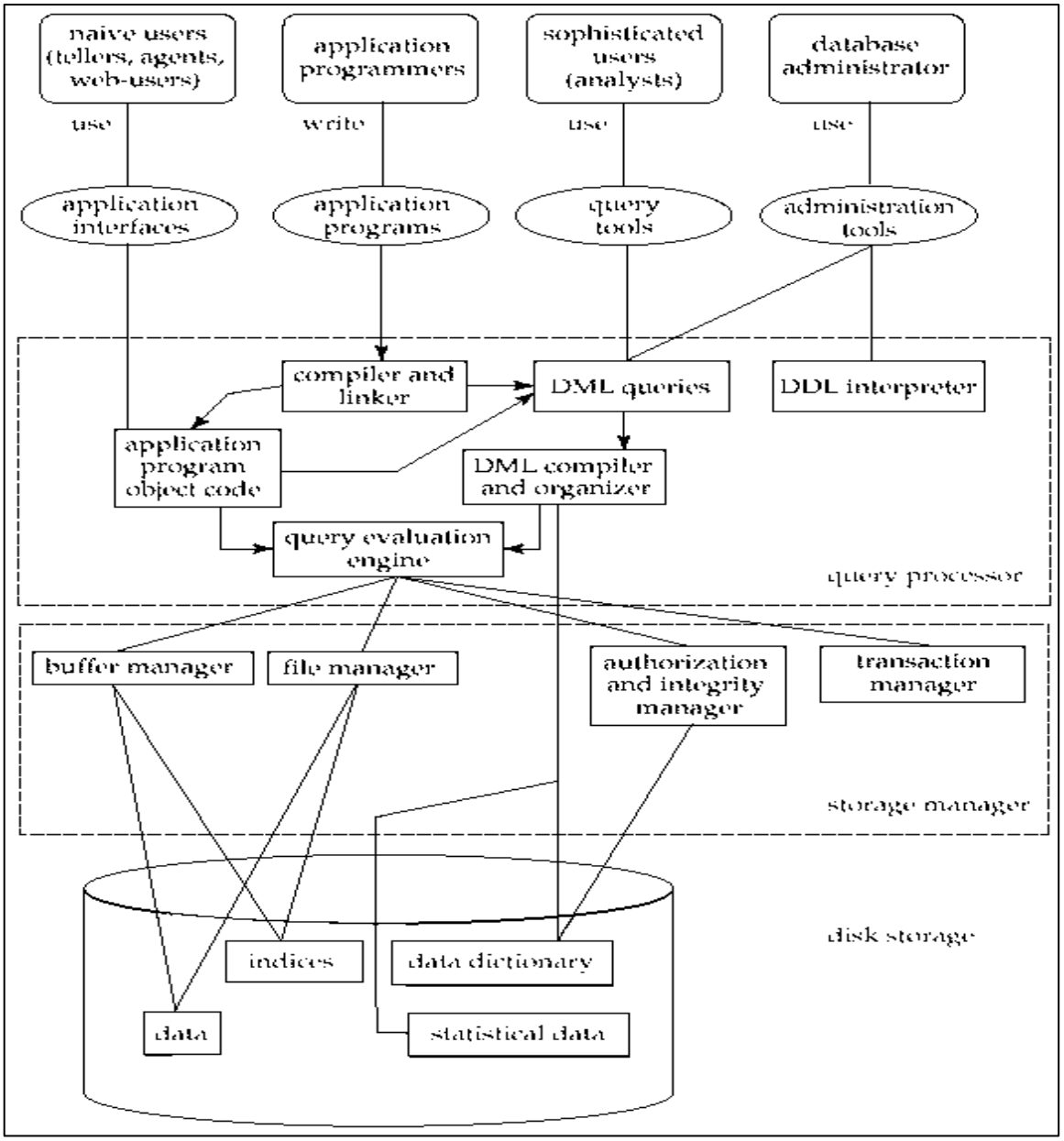
- **Atomicity**: A transaction is a single unit of operation. You either execute it entirely or do not execute it at all. There cannot be partial execution.
- **Consistency**: Once the transaction is executed, it should move from one consistent state to another.

- **Isolation:** Transaction should be executed in isolation from other transactions (no Locks). During concurrent transaction execution, intermediate transaction results from simultaneously executed transactions should not be made available to each other. (Level 0,1,2,3)
- **Durability:** · After successful completion of a transaction, the changes in the database should persist. Even in the case of system failures.

DATABASE SYSTEMS STRUCTURE:

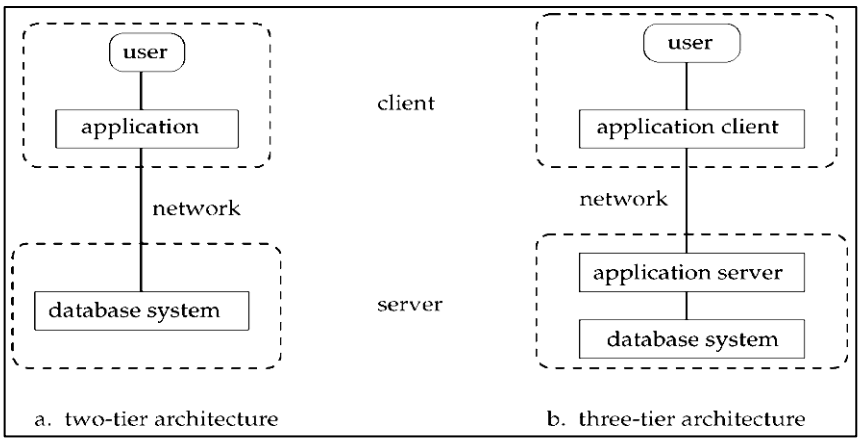
The functional components of a database system can be broadly divided into

- Query processor – The query processor components include
    - DML compiler, which translates DML statements in a query language into low-level instructions that the query evaluation engine understands.
    - Embedded DML precompiler, which converts DML statements embedded in an application program to normal procedure calls in the host language.
    - DDL interpreter, which interprets DDL statements and records them in the data dictionary.
    - Query evaluation engine, which executes low-level instructions generated by the DML compiler.



- Storage manager – The storage manager components provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system. The components include
    - Authorization and integrity manager, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
    - Transaction manager, which ensures that the database remains in a consistent state despite system failures.
    - File manager, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
    - Buffer manager, which is responsible for fetching data from disk storage into main memory.
- In addition, several data structures are required as part of the physical system implementation:
    - Data files, which store the database itself.
    - Data dictionary, which stores metadata about the structure of the database.
    - Indices, which provide fast access to data items that hold particular values.
    - Statistical data, which store statistical information about the data in the database.

- **Application Architecture:** Database applications are usually partitioned into two or three parts.



- In two-tier architecture, the application is partitioned into a component that resides at the client machine, which invokes database system functionality at the server machine through query language statements. E.g. client programs using ODBC/JDBC to communicate with a database.
- In three-tier architecture, the client machine acts as a front end and does not contain any direct database calls. It communications with an application server through a forms interface. The application server in turn communicates with a database system to access data. E.g. web-based applications and applications built using "middleware".

DATABASE SCHEMA:

A database schema defines how data is organized within a relational database; this is inclusive of logical constraints such as, table names, fields, data types, and the relationships between these entities. Schemas commonly use visual representations to communicate the architecture of the database, becoming the foundation for an organization‟s data management discipline. This process of database schema design is also known as data modeling.

These data models serve a variety of roles, such as database users, database administrators, and programmers. For example, it can help database administrators manage normalization processes to avoid data duplication. Alternatively, it can enable analysts to navigate these data structures to conduct reporting or other valuable business analyses. These diagrams act as valuable documentation within the database management system (DBMS), ensuring alignment across various stakeholders.

KEYS:

KEYS in DBMS is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table). They allow you to find the relation between two tables. Keys help you uniquely identify a row in a table by a combination of one or more columns in that table. Key is also helpful for finding unique record or row from the table. Database key is also helpful for finding unique record or row from the table.

Why we need a Key?

Here are some reasons for using sql key in the DBMS system.

- Keys help you to identify any row of data in a table. In a real-world application, a table could contain thousands of records. Moreover, the records could be duplicated. Keys ensure that you can uniquely identify a table record despite these challenges.
- Allows you to establish a relationship between and identify the relation between tables
- Help you to enforce identity and integrity in the relationship.

TYPES OF KEYS:

There are mainly seven different types of Keys in DBMS and each key has its different functionality:

- **Super Key -** A super key is a group of single or multiple keys which identifies rows in a table.
- **Primary Key -** is a column or group of columns in a table that uniquely identify every row in that table.

- **Candidate Key -** is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes.
- **Alternate Key -** is a column or group of columns in a table that uniquely identify every row in that table.
- **Foreign Key -** is a column that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity.
- **Compound Key -** has two or more attributes that allow you to uniquely recognize a specific record. It is possible that each column may not be unique by itself within the database.
- **Composite Key -** An artificial key which aims to uniquely identify each record is called a surrogate key. These kind of key are unique because they are created when you don't have any natural primary key.
- **Surrogate Key -** An artificial key which aims to uniquely identify each record is called a surrogate key. These kind of key are unique because they are created when you don't have any natural primary key.

Primary key example:
CREATE TABLE Persons (ID int NOT NULL, LastName varchar(255) NOT NULL, FirstName varchar(255), Age int, PRIMARY KEY (ID));

Create Primary Key (ALTER TABLE statement) Syntax:
The syntax to create a primary key using the ALTER TABLE statement in SQL is: ALTER TABLE table_name
ADD CONSTRAINT constraint_name PRIMARY KEY
(column1, column2, column_n); FOREIGN KEY on CREATE
TABLE
The following SQL creates a FOREIGN KEY on the "PersonID" column when the "Orders" table is created:
CREATE TABLE Orders (OrderID int NOT NULL, OrderNumber int NOT NULL, PersonID int, PRIMARY KEY (OrderID), FOREIGN KEY (PersonID) REFERENCES Persons(PersonID));

CODD'S RULES:

Rule 1 : The Information Rule:

- "All information in a relational data base is represented explicitly at the logical level and in exactly one way - by values in tables."
- Everything within the database exists in tables and is accessed via table access routines.

Rule 2 : Guaranteed access Rule:

- "Each and every datum (atomic value) in a relational data base is guaranteed to be logically accessible by resorting to a combination of table name, primary key value and column name."
- To access any data-item you specify which column within which table it exists, there is no reading of characters 10 to 20 of a 255 byte string.

Rule 3 : Systematic treatment of null values:

- "Null values (distinct from the empty character string or a string of blank characters and distinct from zero or any other number) are supported in fully relational DBMS for representing missing information and inapplicable information in a systematic way,independent of data type."
- If data does not exist or does not apply then a value of NULL is applied, this is understood by the RDBMS as meaning non-applicable data.

Rule 4 : Dynamic on-line catalog based on the relational model:

- "The data base description is represented at the logical level in the same way as- ordinary data, so that authorized users can apply the same relational language to its interrogation as they apply to the regular data."
- The Data Dictionary is held within the RDBMS, thus there is no-need for off-line volumes to tell you the structure of the database.

Rule 5 : Comprehensive data sub-language Rule:

- "A relational system may support several languages and various modes of terminal use (for example, the fill-in-the-blanks mode). However, there must be at least one language whose statements are expressible, per some well-defined syntax, as character strings and that is comprehensive in supporting all the following items:
  - Data Definition
  - View Definition
  - Data Manipulation (Interactive and by program).
  - Integrity Constraints
  - Authorization.
- Every RDBMS should provide a language to allow the user to query the contents of the RDBMS and also manipulate the contents of the RDBMS.

Rule 6 : .View updating Rule:

- "All views that are theoretically updateable are also updateable by the system."
- Not only can the user modify data, but so can the RDBMS when the user is not logged-in.

Rule 7 : High-level insert, update and delete:

- "The capability of handling a base relation or a derived relation as a single operand applies not only to the retrieval of data but also to the insertion, update and deletion of data."
- The user should be able to modify several tables by modifying the view to which they act as base tables.

Rule 8 : Physical data independence:

- "Application programs and terminal activities remain logically unimpaired whenever any changes are made in either storage representations or access methods."
- The user should not be aware of where or upon which media data-files are stored

Rule 9 : Logical data independence:

- "Application programs and terminal activities remain logically unimpaired when information-preserving changes of any kind that theoretically permit un-impairment are made to the base tables."
- User programs and the user should not be aware of any changes to the structure of the tables (such as the addition of extra columns).

Rule 10 : Integrity independence:

- "Integrity constraints specific to a particular relational data base must be definable in the relational data sub-language and storable in the catalog, not in the application programs."
- If a column only accepts certain values, then it is the RDBMS which enforces these constraints and not the user program, this means that an invalid value can never be entered into this column, whilst if the constraints were enforced via programs there is always a chance that a buggy program might allow incorrect values into the system.

Rule 11 : Distribution independence:

- "A relational DBMS has distribution independence."
- The RDBMS may spread across more than one system and across several networks, however to the end-user the tables should appear no different to those that are local.

Rule 12 : Non-subversion Rule:

- "If a relational system has a low-level (single-record-at-a-time) language, that low level cannot be used to subvert or bypass the integrity Rules and constraints expressed in the higher level relational language (multiple-records-at-a-time)."

RELATIONAL DATABASES:

A relational database is a type of database that stores and provides access to data points that are related to one another. Relational databases are based on the relational model, an intuitive, straightforward

way of representing data in tables. In a relational database, each row in the table is a record with a unique ID called the key. The columns of the table hold attributes of the data, and each record usually has a value for each attribute, making it easy to establish the relationships among data points.

Relational schema:

A relational schema specifies the relation‟ name, its attributes and the domain of each attribute. If R is the name of a relation and A1,A2,… and is a list of attributes representing R then R(A1,A2,…,an) is called a relational schema. Each attribute in this relational schema takes a value from some specific domain called domain(Ai).

Example:

PERSON(PERSON_IDinteger,NAME: STRING,AGE:INTEGER,ADDRESS:string)

Total number of attributes in a relation denotes the degree of a relation.since the PERSON relation schemea contains four attributes ,so this relation is of degree 4.

STRUCTURED QUERY LANGUAGE (SQL):

SQL stands for Structured Query Language.SQL lets you access and manipulate databases . In 1986, the American National Standards Institute (ANSI) and the International organization for standardization (ISO) published an SQL Standard, called SQL-86. IBM published its own corporate SQL standard, the systems Application Architecture Database interface (SAA-SQL) in 1987. ANSI Published on extended standard for SQL, SQL-89, in 1989. The next version of the standard was SQL-92, and the most recent version is SQL:1999.

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

The SQL language has several parts:

- Data-Definition language (DDL): The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- Interactive data – manipulation language (DML): The SQL DML includes a query language based on both the relational algebra and the tuple relational calculus. It includes also commands to insert tuples into, delete tuples from, and modify tuples in the database.
- View definition: The SQL DDL includes commands for defining views.

- Transaction control: SQL includes commands for specifying the beginning and ending of transactions.
- Embedded SQL and dynamic SQL: Embedded and SQL dynamic SQL define how SQL statements can be embedded within general– purpose programming languages, such as C, C++, Java, PL/I, Cobol, Pascal, and FORTRAN.
- Integrity: The SQL DDL includes commands for specifying integrity constraints that the data stored in the databases must satisfy. Updates that violated integrity constraints are disallowed.
- Authorization: The SQL DDL includes commands for specifying access rights to relations and views.

Data Definition:

DDL is short name of Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

1)**CREATE** - to create a database and its objects like (table, index, views, store procedure, function, and triggers)

Syntax

CREATE TABLE *table_name* (*column1datatype,column2datatype,column3datatype,.....*);

The column parameters specify the names of the columns of the table.The datatype parameter specifies the type of data the column can hold (e.g. varchar, integer, date, etc.).

The following example creates a table called "Persons" that contains five columns: PersonID, LastName, FirstName, Address, and City:

Example

CREATE TABLE Persons(PersonIDint,LastNamevarchar(255),FirstNamevarchar(255),

Addressvarchar(255),Cityvarchar(255));

2)**ALTER** - alters the structure of the existing database

SYNTAX

ALTER TABLE table_name ADD column_name datatype;

Example

ALTER TABLE Customers ADD Email varchar(255);

3)**DROP** - delete objects from the database

Syntax:

DROP TABLE *table_name*;

Example:

DROP TABLE Shippers;

4) **TRUNCATE** - remove all records from a table, including all spaces allocated for therecords are removed

Syntax:

TRUNCATE TABLE Table name;

Difference between Drop and Truncate command

5)**COMMENT** - add comments to the data dictionary

Single line comments start with --.

Any text between -- and the end of the line will be ignored (will not be executed).The following example uses a single-line comment as an explanation:

```
--Select all:
SELECT * FROM Customers;
```

The following example uses a single-line comment to ignore the end of a line:

```
SELECT * FROM Customers -- WHERE City='Berlin';
```

The following example uses a single-line comment to ignore a statement:

```
--SELECT * FROM Customers;
SELECT * FROM Products;
```

6)**RENAME** - rename an object

The RENAME TABLE statement is used to change the table name.

Syntax:

ALTER tableName RENAME TO

newTableName;

Example:

RENAME EMPLOYEE1 TO EMPLOYEE2;

Or

ALTER EMPLOYEE1 RENAME TO EMPLOYEE2;

Domain types:

Domain Types in SQL: The SQL standard supports a variety of built-in domain types,including:

- char (n): A fixed-length character string with user-specified length n.
- varchar (n): A variable-length character string with user-specified

maximum length n.

- int: An integer (a finite subset of the integers that is machine dependent).
- smallint: A small integer (a machine-dependent subset of the integer domain type).
- numeric (p, d): A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point.
- real, double precision: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- float (n): A floating-point number, with precision of at least n digits.
- date: A calendar date containing a (four-digit) year, month, and day of the month.
- time: The time of day, in hours, minutes, and seconds. A variant, time (p), can be used to specify the number of fractional digits for seconds. It is also possible to store time zone information along with the time.
- timestamp: A combination of date and time. A variant, timestamp (p), can be used to specify the number of fractional digits for seconds (the default here being 6).

Date and time values can be specified like this:

date „2008-02-25“

time “09:30:00“

timestamp „2008-02-25 10:29:01.45“

An expression of the form cast e as t to convert a character string (or string valued expression) e to the type t, where t is one of date, time, or timestamp. To extract individual fields of a date or time value d, extract (field from d) can be used, where field can be one of year, month, day, hour, minute, or second.

SQL allows comparison operations of all the domains, and it allows both arithmetic and comparison operations on the various numeric domains. SQL also provides a data type called interval, and it allows computations based on dates and times and on intervals.

SQL allows the domain declaration of an attribute to include the specification not null and thus prohibits the insertion of a null value for this attribute.

Structure of SQL Queries:

A relational database consists of a collection of relations, each of which is assigned a unique name. The basic structure of an SQL expression consists of three clauses: select, from, and where. A typical SQL query has the form

select A_1, A_2, …, A_n from r_1, r_2, …, r_m where P

Each A_i presents an attribute, and each r_i a relation. P is a predicate.

- The select Clause: The result of an SQL query is a relation. Consider a simple query

select branch–name from loan

The result is a relation consisting of a single attribute with the heading branch–name. SQLallows duplicates in relations as well as in the results of SQL expressions.

To force the elimination of duplicates, insert the keyword distinct after select. To display onlydistinct branch–names, the above query can be rewritten as

select distinct branch–name from loan

The keyword all is used to specify explicitly that duplicates are not

removed.select all branch–name from loan

A **select clause** of the form select * indicates that all attributes of all relations appearing in thefrom clause are selected.

The select clause may also contain arithmetic expressions involving the operators +, –, *, and
/ operating on constants or attributes of tuples.

SQL also provides special data types, and allows several arithmetic functions to operate onthese types.

- **The where Clause:** Consider the query "Find all loan numbers for loans made at the Perry ridge branch with loan amounts greater than $1200". This query can be written in SQL asemd

select loan–number from loan where branch name = "Perry ridge" and amount > 1200

SQL uses the logical connections and, or, and not – rather than the mathematical symbols
$\wedge$,$\vee$, and $\neg$ – in the where clause. The operands of the logical connectives can

be expressionsinvolving the comparison operators, <, <=, >, >=, =, and <>.

SQL includes a between comparison operator to simplify where clauses that specify that a value be less than or equal to some value and greater than or equal to some other value. The query to find the loan number of those loans with loan amounts between $90,000 and
$100,000 is

select loan–number from loan where amount between 90000 and

100000instead of

select loan–number from loan where amount <= 100000 and amount

>= 90000Similarly, the not between comparison operator can also

be used.

- The from Clause: The from clause by itself defines a Cartesian product of the relations in the clause. Consider the query "Find the customer names, loan numbers and loan amounts for all loans at the Perry ridge branch". Assume that customer–name, loan–number attributes are in borrower relation and loan– number, branch–name, amount attributes are in loan relation. In SQL, this query can be written as

select customer–name, borrower.loan–number, amount from borrower, loan

where borrower.loan–number = loan.loan–number and branch–name = "Perry ridge".
SQL uses the notation relation–name.attribute–name to avoid ambiguity in cases where an attribute appears in the schema of more than one relation.

- The Rename Operation: It is known that the names of the attributes in the result are derived from the names of the attributes in the relations in the from clause. However SQL provides a way for renaming the attributes of a result relation. It uses the as clause, taking the form:

old–name as new–name

The following query replaces the attribute name loan–number with the name loan–id while finding the customer names, loan numbers and loan amount those who have a loan from a bank.

select customer–name, borrower.loan–number as loan–id,

amount from borrower, loan where borrower.loan–number =

loan.loan–number

- Tuple Variables: A tuple variable in SQL must be associated with a particular relation. Tuple variables are defined in the from clause by way of the as clause. The query “For all customers who have a loan from the bank, find their names, loan numbers, and loan amount”, can be rewritten as

select customer–name, T.loan–number, S.amount from

borrower as T,loan as S where T.loan–number = S.loan–

number

A tuple variable is defined in the from clause by placing it after the name of the relation withwhich it is associated, with the keyword as in between (the keyword as is optional).

Tuple variables are most useful for comparing two tuples in the same relation. The followingexample finds the names of all branches that have assets greater than at least one branch located in Brooklyn.

select distinct T.branch–name from branch as T,

branch as Swhere T.assets > S.assets and S.branch-

city = „Brooklyn“

- String Operations: SQL specifies strings by enclosing them in single quotes. A single quote character that is part of a string can be specified by using two single quote characters; for example the string “It“s right” can be specified by „It““s right“.

The most commonly used operation on strings is pattern matching using the operator like.Patterns are described by using two special characters:

- Percent (%): The % character matches any substring.
- Underscore (_): The _ character matches any character.

Patterns are case sensitive; i.e., uppercase characters do not match lowercase characters, orvice versa. To illustrate pattern matching, consider the following examples:

- „Perry%“ matches any string beginning with “Perry”.

- „%idge%“ matches any string containing “idge” as a substring, for example,
  „Perryridge“, „Rock Ridge“, „Mianus Bridge“, and „Ridgeway“.

- „___“ matches any string of exactly three characters.
- „___%“ matches any string of at least three characters.

SQL expresses patterns by using the like comparison operator. Consider the query “Find the names of all customers whose street address includes the substring „Main“”. This query can be written as

select customer-name from

customer where customer-street

like „%Main%“

For patterns to include the special pattern characters (% and _), SQL allows the specification of an escape character. The escape character is used immediately before a special pattern character to indicate that the special character is to be treated like a normal character. To illustrate, consider the following patterns, which use a backslash (\) as the escape character:

- like „ab\%cd%“ escape „\“ matches all strings beginning with “ab%cd”.
- like „ab\\cd%“ escape „\“ matches all strings beginning with “ab\cd”.

SQL allows us to search for mismatches instead of matches by using the not like comparison operator. SQL also permits a variety of functions on character strings, such as concatenation (using “||”), extracting substrings, finding the length of strings, converting between uppercase and lowercase, and so on. SQL:1999 also offers a *similar to* operation, which provides more powerful pattern matching than the *like* operation.

- Ordering the Display of Tuples: The order by clause causes the tuples in the resultof a query to appear in sorted order. By default, the order by clause lists items in ascending order. To specify the sort order, use desc for descending order or asc for ascending order. Ordering can also be performed on multiple attributes.

Suppose to list the entire loan relation in descending order of amount, and if several loans have the same amount, then to order them in ascending order by loan number, the query can be written as

select * from loan order by amount desc, loan-number asc

Since sorting a large number of tuples may be costly, it should be done only when necessary.

- Duplicates: SQL formally defines not only what tuples are in the result of a query,but also how many copies of each of those tuples appear in the result. The duplicate semantics of an SQL query can

be defined using multiset versions of therelational operators.

Modifications of the database:

Referential integrity is a condition, which ensures that a value that appears in one relation fora given set of attributes also appears for a certain set of attributes in another relation.

Let r_1 (R_1) and r_2 (R_2) be relations with primary keys K_1 and K_2, respectively. A subset α of R_2 is a foreign key referencing K_1 in relation r_1 if it is required that, for every tuple t_2 in r_2, there must be a tuple t_1 in r_1 such that $t_1[K_1] = t_2[\alpha]$. Requirements of this form are called referential integrity constraints or subset dependencies, because the referential integrity constraint can be written as $\Pi_\alpha (r_2) \subseteq \Pi_{K1} (r_1)$. The symbol Π is used to indicate the projection operation.

Database modifications can cause violations of referential integrity. The following list of tests must be made for each type of database modification to preserve the following integrity constraint:

$\Pi_\alpha (r_2) \subseteq \Pi_K (r_1)$

- **Insert.** If a tuple t_2 is inserted into r_2, the system must ensure that there is a tuple t_1
  in r_1 such that $t_1[K] = t_2[\alpha]$. i.e. $t_2[\alpha] \in \Pi_K (r_1)$.
- **Delete.** If a tuple t_1 is deleted from r_1, the system must compute the set of tuples inr_2 that reference t_1: $\sigma_{\alpha = t1[K]} (r_2)$. If this set is not empty, either the delete command is rejected as an error, or the tuples that reference t1 must themselves be deleted.
- **Update.** Consider two cases for update: updates to the referencing relation (r_2), and updates to the referenced relation (r_1).
  - If a tuple t_2 is updated in relation r_2, and the update modifies values for the foreignkey α, then the system must ensure that $t_2'[\alpha] \in \Pi_K (r_1)$, where t_2' denotes the new value of tuple t_2.
  - If a tuple t_1 is updated in r_1, and the update modifies values for the primary key (K), then the system must compute $\sigma_{\alpha = t1[K]} (r_2)$ using the old value of t_1 (the value before the update is applied). If this set is not empty, the update is rejected as an error, or the update is cascaded in a manner similar to delete.

Foreign keys can be specified as part of the SQL *create table* statement by using the foreign key clause. By default, a foreign key references the primary

key attributes of the referenced table. SQL also supports a version of the *references* clause where a list of attributes of the referenced relation can be specified explicitly. The specified list of attributes must be declared as a candidate key of the referenced relation.

Consider the definition of an integrity constraint on the relation account with the referenced relation branch:

create table branch (branch-name char (15), branch-city char (30), assets integer, primary key(branch-name), check (assets >= 0))

create table account (account-number char (10), branch-name char (15), balance integer, primary key (account-number), foreign key (branch-name) references branch on delete cascade on update cascade, check (balance >= 0))

When a referential integrity constraint is violated, the normal procedure is to reject the actionthat caused the violation.

Because of the clause *on delete cascade* associated with the foreign key declaration, if a delete of a tuple in branch results in this referential integrity constraint being violated, the system does not reject the delete. Instead, the delete "cascades" to the account relation, deleting the tuple that referes to the branch that was deleted.

Similarly, the system does not reject an update to a field referenced by the constraint if it violates the constraint; instead, the system updates the field branch-name in the referencing tuples in account to the new value.

SQL also allows the foreign key clause to specify actions other than cascade, if the constraintis violated. The referencing field branch-name can be set to null (by using set null in place ofcascade), or to the default value for the domain (by using set default).

If there is a chain of foreign key dependencies across multiple relations, a deletion or update at one end of the chain can propagate across the entire chain.

When the foreign key constraint on a relation references the same relation, then if a cascading update or delete causes a constraint violation that cannot be handled by a further cascading operation, the system aborts the transaction, and all the changes caused by the transaction andits cascading actions are undone.

Set Operations:

The SQL operations union, intersect, and except operate on relations and correspond to the relational algebra operations $\cup$, $\cap$, and $-$. The relations

participating in the operations must have the same set of attributes.

1) **The Union Operation**: The union operation automatically eliminates duplicates. The following query finds all customers having a loan, an account, or both at the bank. If a customer has several accounts or loans (or both) at the bank, then that customer will appear only once in the result.

(select customer-name from depositor) union (select customer-name

from borrower)If all duplicates are to be retained, then write union all

in place of union.

(select customer-name from depositor) union all (select customer-name from borrower)

The number of duplicate tuples in the result is equal to the total number of duplicates that appear in both depositor and borrower. Thus, if a customer has three accounts and two loans at the bank, then there will be five tuples with the same customer name in the result.

Syntax

SELECT column_name FROM

table1UNION

SELECT column_name FROM table2;

Example:

The First table

ID	NAME
1	Jack
2	Harry
3	Jackson

The Second table

ID	NAME
3	Jackson

4	Stephan
5	David

Union SQL query will be: SELECT * FROM First UNION SELECT * FROM

Second;The resultset table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
4	Stephan
5	David

2) Union All:

Union All operation is equal to the Union operation. It returns the set without removingduplication and sorting the data.

Syntax:

```
SELECT column_name
FROM table1UNION ALL
SELECT column_name FROM table2;
```

Example: Using the above First and Second table.Union All query will be like:

```
SELECT * FROM
FirstUNION ALL
```

SELECT * FROM

Second;The resultset table

will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
3	Jackson
4	Stephan
5	David

3) Intersect:

- It is used to combine two SELECT statements. The Intersect operation returns thecommon rows from both the SELECT statements.
- In the Intersect operation, the number of datatype and columns must be the same.
- It has no duplicates and it arranges the data in ascending order by default.

Syntax

SELECT column_name

FROM table1INTERSECT

SELECT column_name FROM table2;

Example:

Using the above First and Second table.

Intersect query will be:

SELECT * FROM

FirstINTERSECT

SELECT * FROM

Second;The resultset table

will look like:

ID	NAME
3	Jackson

4)EXCEPT OR MINUS:

- It combines the result of two SELECT statements. Minus operator is used to displaythe rows which are present in the first query but absent in the second query.
- It has no duplicates and data arranged in ascending order by default.

Syntax:

```
SELECT column_name
FROM table1 MINUS
SELECT column_name FROM table2;
```

Example

Using the above First and Second table.

Minus query will be:

```
SELECT * FROM
First MINUS
SELECT * FROM
```

Second;The resultset table

will look like:

ID	NAME
1	Jack
2	Harry

Aggregate Functions:

In database management an aggregate function is a function where the values of multiple rows are grouped together as input on certain criteria to form a single value of more significant meaning.

Various Aggregate Functions

1) Count()

2) Sum()

3) Avg()

4) Min()

5) Max()

Now let us understand each Aggregate function with a example:

Id	Name	Salary
1	A	80
2	B	40
3	C	60
4	D	70
5	E	60
6	F	NULL

Count():

Count(*): Returns total number of records .i.e 6.
Count(salary): Return number of Non Null values over the column salary. i.e 5.
Count(Distinct Salary): Return number of distinct Non Null values over the column salary. i.e 4

Sum():

sum(salary): Sum all Non Null values of Column salary i.e., 310.
sum(Distinct salary): Sum of all distinct Non-Null values i.e., 250.

Avg():

Avg(salary) = Sum(salary) / count(salary) = 310/5
Avg(Distinct salary) = sum(Distinct salary) / Count(Distinct Salary) = 250/4

Min():

Min(salary): Minimum value in the salary column except NULL i.e., 40.
Max(salary): Maximum value in the salary i.e., 80.

Null Values:

The term **NULL** in SQL is used to specify that a data value does not exist in the database. It is not the same as an empty string or a value of zero, and it signifies the absence of a value orthe unknown value of a data field.

Some common reasons why a value may be NULL −

- The value may not be provided during the data entry.

- The value is not yet known.

It is important to understand that you cannot use comparison operators such as "=", "<", or ">" with NULL values. This is because the NULL values are unknown and could represent any value. Instead, you must use "IS NULL" or "IS NOT NULL" operators to check if a value is NULL.

Syntax

The basic syntax of **NULL** while creating a

table.SQL> CREATE TABLE

CUSTOMERS(

ID INT NOT NULL,

NAME VARCHAR (20) NOT

NULL,AGE INT NOT

NULL, ADDRESS CHAR (25) ,

SALARY DECIMAL (18,

2),PRIMARY KEY (ID)

);

Here, **NOT NULL** signifies that column should always accept an explicit value of the given data type. There are two columns where we did not use NOT NULL, which means these columns could be NULL.

A field with a NULL value is the one that has been left blank during the record creation.

Example

Let us create a table with the name **CUSTOMERS** in the SQL database using the CREATE statement as shown in the query below −

SQL> CREATE TABLE

CUSTOMERS(ID INT NOT

NULL,

NAME VARCHAR (20) NOT

NULL,AGE INT NOT NULL,

ADDRESS CHAR (25) ,

```
    SALARY  DECIMAL (18,
    2),PRIMARY KEY (ID)
);
```

Let us insert some values into the above created table using the following query −

```
SQL> INSERTINTO CUSTOMERS (ID, NAME, AGE,
     ADDRESS, SALARY)
VALUES(1, 'Ramesh', '32', 'Ahmedabad', 2000);

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(2,
'Khilan', '25', 'Delhi', 1500);

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(3,
'kaushik', '23', 'Kota', 2000);

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(4,
'Chaitali', '25', 'Mumbai', 6500);

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(5,
'Hardik','27', 'Bhopal', 8500);

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(6,'Komal', '22', 'MP', NULL);

INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(7,
'Muffy', '24', 'Indore', NULL);
```

To verify whether the table CUSTOMERS is created or not, use the

following query −SQL> SELECT * FROM CUSTOMERS;

The table is successfully created in the database.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	NULL
7	Muffy	24	Indore	NULL

IS NOT NULL Query

Now, let us try to retrieve the records present in the table that are not null using the **IS NOTNULL** operator −

```
SQL> SELECT ID, NAME, AGE, ADDRESS,
  SALARYFROM CUSTOMERS
  WHERE SALARY IS NOT NULL;
```

Output

The above query would produce the following result −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00

IS NULL Query

Let us try to retrieve the records present in the table that are null using the **IS NULL** operator
–

```
SQL> SELECT ID, NAME, AGE, ADDRESS,
  SALARYFROM CUSTOMERS
  WHERE SALARY IS NULL;
```

Output

The above query would produce the following result −

ID	NAME	AGE	ADDRESS	SALARY
6	Komal	22	MP	NULL
7	Muffy	24	Indore	NULL

Updating NULL values in a table

You can update the NULL values present in a table using the **UPDATE** statement in SQL. To do so, you can use the IS NULL operator in your WHERE clause to select the rows with NULL values and then set the new value using the SET keyword.

Example

Assume the previously created table and let us try to update the NULL value(s) in the presentin the table using the **UPDATE** statement as shown below −

SQL> UPDATE CUSTOMERS SET SALARY = 9000 WHERE SALARY IS NULL;

Output

When you execute the above query, the output is obtained as

follows −Commands completed successfully.

Verification

Let us try to verify whether the specified record(s) in the table is updated or not using the following query −

SQL> SELECT * FROM CUSTOMERS;

On executing the above query, the output is displayed as follows −

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	9000.00
7	Muffy	24	Indore	9000.00

Nested Sub queries:

A nested query is a query that has another query embedded within it. The embedded query iscalled a subquery.

A subquery typically appears within the WHERE clause of a query. It can sometimes appearin the FROM clause or HAVING clause.

Example

Let‟s learn about nested queries with the help of an

example.Find the names of employee who have

regno=103

The query is as follows −

select E.ename from employee E where E.eid IN (select S.eid from salary S whereS.regno=103);

Student table

The student table is created as follows −

create table student(id number(10), name varchar2(20),classID number(10),

marks varchar2(20));

```
Insert into student values(1,'pinky',3,2.4);
Insert into student values(2,'bob',3,1.44);
Insert into student values(3,'Jam',1,3.24);
Insert into student values(4,'lucky',2,2.67);
Insert into student values(5,'ram',2,4.56);
select * from student;
```

Output

You will get the following output −

Id	Name	classID	Marks
1	Pinky	3	2.4
2	Bob	3	1.44
3	Jam	1	3.24
4	Lucky	2	2.67
5	Ram	2	4.56

Teacher table

The teacher table is created as follows −

Example

Create table teacher(id number(10), name varchar(20), subject varchar2(10), classIDnumber(10), salary number(30));

```
Insert into teacher values(1,"bhanu","computer",3,5000);
Insert into teacher values(2,'rekha','science',1,5000);
Insert into teacher values(3,'siri','social',NULL,4500);
Insert into teacher values(4,'kittu','mathsr',2,5500);
select * from teacher;
```

Output

You will get the following output −

Id	Name	Subject	classID	Salary
1	Bhanu	Computer	3	5000
2	Rekha	Science	1	5000
3	Siri	Social	NULL	4500
4	Kittu	Maths	2	5500

Class table

The class table is created as follows −

Example

Create table class(id number(10), grade number(10), teacherID number(10), noofstudentsnumber(10));

insert into class

values(1,8,2,20); insert into

class values(2,9,3,40); insert

into class values(3,10,1,38);

select * from class;

Output

You will get the following output –

ID	Grade	teacherID	No.ofstudents
1	8	2	20
2	9	3	40
3	10	1	38

Now let‟s work on nested queries

Example 1

Select AVG(noofstudents) from class where

teacherID IN(Select id from teacher

Where subject=‟science‟ OR subject=‟maths‟);

Output

You will get the following

output −20.0

Example 2

```
SELECT * FROM
studentWHERE
classID = (
    SELECT id
    FROM class
    WHERE noofstudents = (
      SELECT
      MAX(noofstudents)
      FROM class));
```

Output

You will get the following

output −4|lucky |2|2.67

5|ram |2|4.56

Complex Queries:

Complex queries are often impossible to write as a single SQL block or a union / intersection / difference of SQL blocks. There are two ways of composing multiple SQL blocks to expressa complex query.

Derived Relations: SQL allows a subquery expression to be used in the from clause. If such an expression is used, then the result relation must be given a name, and the attributes can be renamed. This renaming is done using the as clause. Consider the subquery

```
select branch-name,avg (balance) from account group by branch-name)
as result (branch-name, avg-balance);
```

This subquery generates a relation consisting of the names of all branches and

their corresponding average account balances. The subquery result is named result, with the attributes branch-name and avg-balance.

The following query finds the average account balance of those branches where the average account balance is greater then $1200.

(select branch-name, avg-balance

from (select branch-name, avg

(balance)from account

group by branch-name)

as branch-avg (branch-name, avg-

balance)where avg-balance > 1200

The subquery in the from clause computes the average balance, and it result is named asbranch-avg. The attributes of branch-avg can be used directly in the where clause.

The query shown below finds the maximum across all branches of the total balance at eachbranch.

select max (tot-balance)

from (select branch-name, sum

(balance)from account group by

branch-name)

as branch-total (branch-name, tot-balance)

The With Clause: The with clause provides a way of defining a temporary view whose definition is available only to the query in which the with clause occurs. Consider the following query, which selects accounts with the maximum balance; if there are many accounts with the same maximum balance, all of them are selected.

with max-balance (value)

as select max (balance)

from account

select account-number
from account, max-balance

where account.balance = max-balance.value

The with clause makes the query logic clearer; it also permits a view definition to be used inmultiple places within a query.

The following query is to find all branches where the total account deposit is less than theaverage of the total account deposits at all branches.

with branch-total (branch-name, value) as select branch-name, sum (balance)

from account

group by branch-name

with branch-total-avg (value) as select avg (value)

from branch-total select branch-name

from branch-total, branch-total-avg

where branch-total.value < branch-total-avg.value;

Views:

- Views in SQL are considered as a virtual table. A view also contains rows andcolumns.
- To create the view, we can select the fields from one or more tables present inthe database.

A view can either have specific rows based on certain condition or all the rows of a table.

Sample table: Student_Detail

STU_ID	NAME	ADDRESS
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Student_Marks

STU_ID	NAME	MARKS	AGE
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

1. **Creating view**

A view can be created using the **CREATE VIEW** statement. We can create a view from asingle table or multiple tables.

Syntax:

CREATE VIEW view_name AS SELECT column1, column2.....

FROM table_name WHERE condition;

2. **Creating View from a single**

tableQuery:

CREATE VIEW DetailsView AS SELECT NAME, ADDRESS

FROM Student_Details WHERE STU_ID < 4;

Just like table query, we can query the view to view the data. SELECT * FROMDetailsView;

Output:

NAME	ADDRESS
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

3. Creating View from multiple tables

View from multiple tables can be created by simply include multiple tables in the SELECTstatement.

In the given example, a view is created named MarksView from two tables Student_Detailand Student_Marks.

Query:

```
CREATE VIEW MarksView AS
SELECT  Student_Detail.NAME,
        Student_Detail.ADDRESS,
        Student_Marks.MARKS FROM Student_Detail, Student_Mark
WHERE  Student_Detail.NAME  =  Student_Marks.NAME;
```

To display data of View MarksView: SELECT * FROM MarksView;

NAME	ADDRESS	MARKS
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurugram	90

4. Deleting View

A view can be deleted using the Drop View statement.

Syntax

1. DROP VIEW view_name;

Example:

If we want to delete the View **MarksView**, we can do this as:

1. DROP VIEW MarksView;

Uses of a View :

A good database should contain views due to the given reasons:

1. **Restricting data access –**

Views provide an additional level of table security by restricting access to a predetermined set of rows and columns of a table.

2. **Hiding data complexity –**

A view can hide the complexity that exists in a multiple table join.

3. **Simplify commands for the user –**

Views allows the user to select information from multiple tables without requiring the users to actually know how to perform a join.

4. **Store complex queries –**

Views can be used to store complex queries.

5. **Rename Columns –**

Views can also be used to rename the columns without affecting the base tables provided thenumber of columns in view must match the number of columns specified in select statement.Thus, renaming helps to to hide the names of the columns of the base tables.

6. **Multiple view facility –**

Different views can be created on the same table for different users.

Joined relations:

Join is a combination of a Cartesian product followed by a selection process. A Join operation pairs two tuples from different relations, if and only if a given join condition is satisfied.

We will briefly describe various join types in the following

sections.Example:

EMPLOYEE

EMP_CODE	EMP_NAME
101	Stephan
102	Jack
103	Harry

SALARY

EMP_CODE	SALARY
101	50000
102	30000
103	25000

1. Operation: (EMPLOYEE ⋈ SALARY)

Result:

PlayNext

Unmute

Current TimeÂ 0:00

/

DurationÂ 18:10

Loaded:

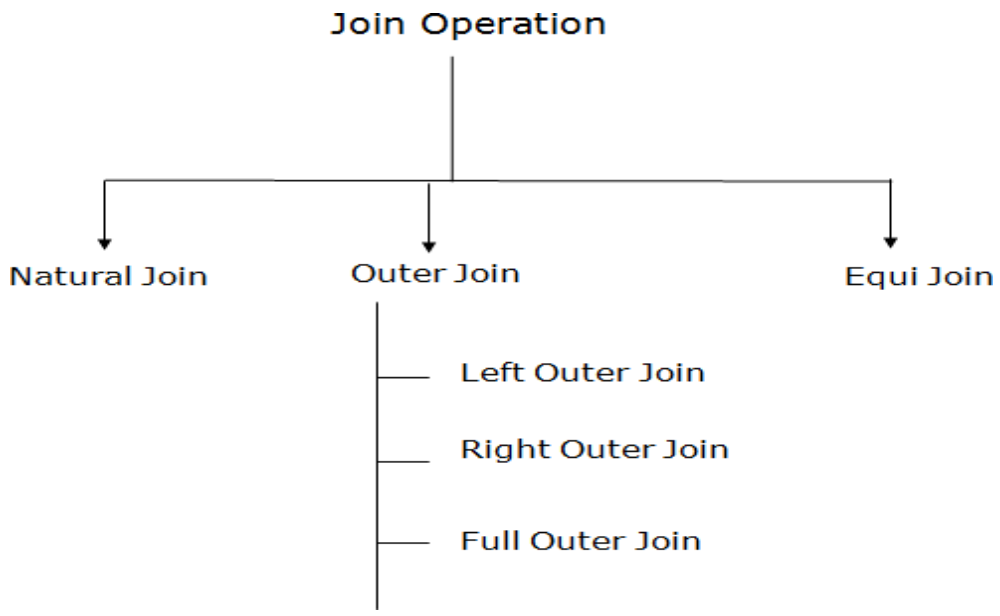
0.37%Â

Fullscreen

Backward Skip 10sPlay VideoForward Skip 10s

EMP_CODE	EMP_NAME	SALARY
101	Stephan	50000
102	Jack	30000
103	Harry	25000

Types of Join operations:



1. **Natural Join:**
   - A natural join is the set of tuples of all combinations in R and S that are equal on theircommon attribute names.
   - It is denoted by ⋈.

Example: Let's use the above EMPLOYEE table and SALARY table:

Input:

1. ∏EMP_NAME, SALARY (EMPLOYEE ⋈ SALARY)

Output:

EMP_NAME	SALARY
Stephan	50000
Jack	30000
Harry	25000

2.Outer Join:

The outer join operation is an extension of the join operation. It is used to deal with missinginformation.

Example:

EMPLOYEE

EMP_NAME	STREET	CITY
Ram	Civil line	Mumbai
Shyam	Park street	Kolkata
Ravi	M.G. Street	Delhi
Hari	Nehru nagar	Hyderabad

FACT_WORKERS

EMP_NAME	BRANCH	SALARY
Ram	Infosys	10000
Shyam	Wipro	20000
Kuber	HCL	30000
Hari	TCS	50000

Input:

1. (EMPLOYEE ⋈ FACT_WORKERS)

Output:

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru nagar	Hyderabad	TCS	50000

An outer join is basically of three types:

a. Left outer join
b. Right outer join
c. Full outer join

a. Left outer join:

- Left outer join contains the set of tuples of all combinations in R and S that are equalon their common attribute names.
- In the left outer join, tuples in R have no matching tuples in S.
- It is denoted by ⟕.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

1. EMPLOYEE ⟕ FACT_WORKERS

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL

b. Right outer join:

- Right outer join contains the set of tuples of all combinations in R and S that are equalon their common attribute names.
- In right outer join, tuples in S have no matching tuples in R.
- It is denoted by ⟖.

Example: Using the above EMPLOYEE table and FACT_WORKERS Relation

Input:

1. EMPLOYEE ⟖ FACT_WORKERS

Output:

EMP_NAME	BRANCH	SALARY	STREET	CITY
Ram	Infosys	10000	Civil line	Mumbai
Shyam	Wipro	20000	Park street	Kolkata
Hari	TCS	50000	Nehru street	Hyderabad
Kuber	HCL	30000	NULL	NULL

c. Full outer join:

- Full outer join is like a left or right join except that it contains all rows from bothtables.
- In full outer join, tuples in R that have no matching tuples in S and tuples in S thathave no matching tuples in R in their common attribute name.
- It is denoted by ⟗.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

1. EMPLOYEE ⟗ FACT_WORKERS

Output:

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL
Kuber	NULL	NULL	HCL	30000

3.Equi join:

It is also known as an inner join. It is the most common join. It is based on matched data asper the equality condition. The equi join uses the comparison operator(=).

Example:

CUSTOMER RELATION

CLASS_ID	NAME
1	John
2	Harry
3	Jackson

PRODUCT

PRODUCT_ID	CITY
1	Delhi
2	Mumbai
3	Noida

Input:

2. CUSTOMER ⋈ PRODUCT

Output:

CLASS_ID	NAME	PRODUCT_ID	CITY
1	John	1	Delhi
2	Harry	2	Mumbai
3	Harry	3	Noida